



INFORME TECNICO ANALITICO

Paradigmas y Lenguajes



INTEGRANTES:

Buyatti Nadia Antonella,
Lovato Buss Cynthia,
Ibalo Pereyra Agustina Beatriz

GRUPO: N°15

TEMA: Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas

LENGUAJE: Scheme

PROFESORES:

Monzón Ricardo
Mascazzini Matias
Ringa Mónica
Rodriguez Leandro

AÑO: 2026

INDICE

Introducción.....2

Desarrollo. Diseño Funcional3

Desarrollo. Justificación de Librerías.....6

Desarrollo. Bitácora de Errores.....7

Desarrollo. Respuestas Teóricas. Fase 3.....10

Conclusión.....12

Bibliografía.....13

INTRODUCCION

En este trabajo integrador, resolvimos los aspectos de un sistema de trafico urbano, partimos de una solución esquemática y luego se procedió a traducirlo en lenguaje LISP utilizando el paradigma funcional.

Evalúamos diferentes alternativas para abordar el problema, realizando casos de prueba para verificar la correcta ejecución, como así también la mejor opción para solucionarlos.

Realizamos búsquedas de información acerca de bibliotecas, modularización de funciones y de lenguajes nuevos, lo que fomenta a la adquisición de nuevos conocimientos y a las diferentes alternativas para gestionar un problema.

DESARROLLO

DISEÑO FUNCIONAL ADOPTADO:

Para el diseño del sistema de control de semáforos, se decidió estructurar el código mediante funciones puras y de impacto no destructivo para resolver toda la lógica del problema.

Se priorizó la modularización del programa, dividiéndolo en funciones chicas y específicas, evitando el uso excesivo de variables globales o funciones que modifiquen los datos originales. Para cumplir con lo solicitado, se aplicaron funciones predicado, evaluaciones booleanas, y condicionales simples (if) y múltiples (cond).

Explicación breve del diseño de cada Requerimiento:

Requerimiento 1

La función es-transicion-permitida recibe como parámetro el color actual y el color destino. Implementa funciones predicado y evaluación booleana, con el objetivo de validar que el flujo de luces respete la secuencia. Si se encuentra con una transición inválida, indica la acción-por-defecto.

Requerimiento 2

Las funciones ubicar-fase y timer resuelven el estado sincrónico del semáforo. Para la validación del Tiempo Unix ingresado, la función timer utiliza un condicional simple (if) junto con integerp para asegurar que el dato sea un número entero. A partir de esta entrada, se aplica la función matemática mod para obtener el segundo actual dentro del ciclo. Luego, ese resultado es procesado por la función ubicar-fase mediante un condicional múltiple (cond), el cual determina qué fase de luz corresponde a ese momento.

Requerimiento 3

Para la función es-color-valido utilizamos una función predicado utilizando un or y la función equal para determinar si el color coincide con los colores del sistema, devolviendo verdadero o falso. En la función informe, se utiliza un condicional simple (if), en donde recibe tres parámetros, de los cuales se valida con un and, en caso de que los datos sean correctos, se imprime el formato con la función format t.

Requerimiento 4

Las funciones duración-ciclo y recomendación-ciclo, clasifican los datos de entrada en categorías y devuelve las recomendaciones necesarias. Las funciones reciben parámetros, en donde se utiliza un cond como condicional múltiple y se procede a la validación del dato con un or. Con un and indicamos la condición que debe cumplir, en donde, sino cumple nada de lo anterior, continúa con el T del cond y realiza una lista.

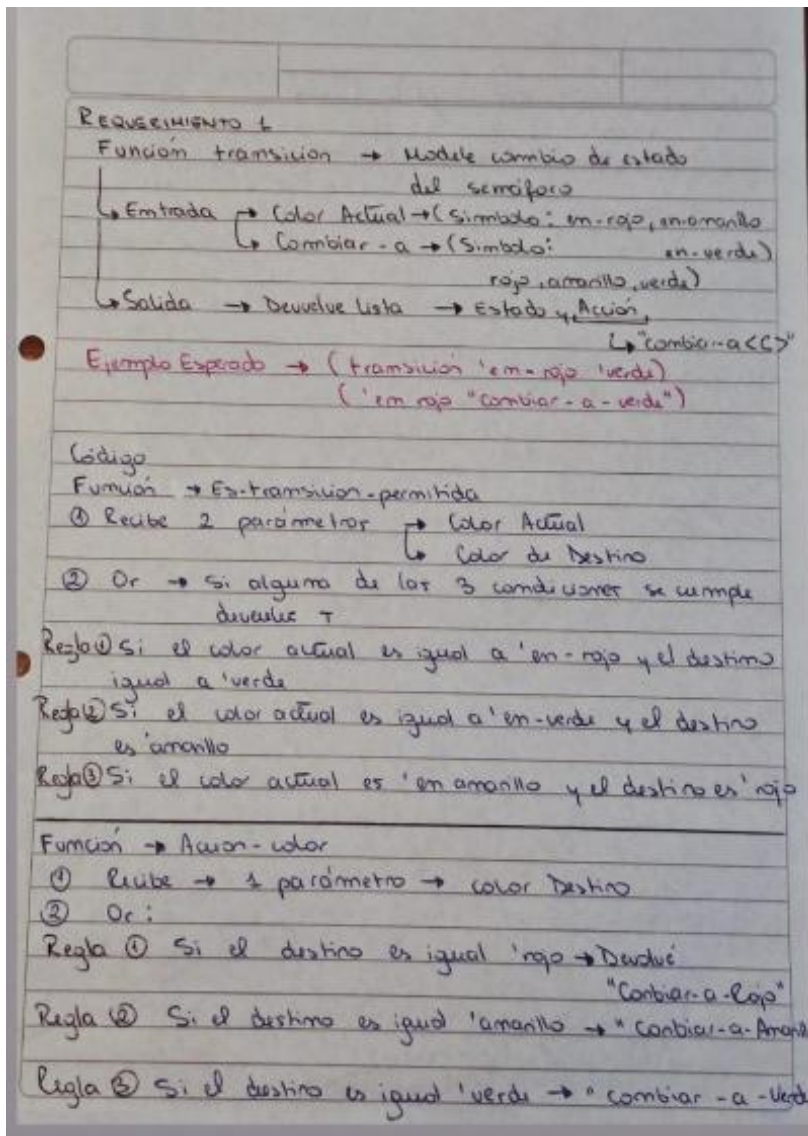
Requerimiento 5

La función ciclos-por-tiempo toma una cantidad de minutos como parámetro, en donde se emplea un condicional (if) junto con un and para comprobar que el dato sea de tipo numérico y que mantenga un valor positivo. Si la validación es correcta, los minutos se transforman a segundos y se procesan con la función truncate para obtener la cantidad exacta de ciclos enteros que entran en dicho periodo, retornando error si los parámetros de entrada no coinciden.

Requerimiento 6

Las funciones ciclos-por-hora y resto-hora operan mediante 3600 segundos, en donde obtienen las vueltas completas y los segundos sobrantes usando truncate y mod, permitiendo que las funciones de cálculo por color determinen los segundos exactos mediante operaciones aritméticas y las funciones min y max para repartir lo que sobra. Posteriormente, la función calcular-porcentaje traduce estos tiempos a valores porcentuales con la función float y la función principal informe-distribucion-hora junta toda la información para entregar una lista definitiva con los porcentajes de cada luz. En este requerimiento se optó por la modularización de las funciones para luego integrarlas.

IMAGEN DESCRIPTIVA



Realizamos un esquema de todos los requerimientos como primer paso y luego fuimos realizando las funciones con las condiciones adecuadas.

JUSTIFICACION DE LIBRERIAS

Para la Fase 2 del proyecto, se decidió incorporar la librería externa local-time, ya que el sistema requería procesar marcas de tiempo en formato Unix Epoch y transformarlas en un formato de fecha y hora que fuera comprensible para un operador humano.

Al seleccionar local-time, se pudo resolver este problema de forma eficiente utilizando funciones preparadas para esta tarea, como unix-to-timestamp, que convierte el número entero de segundos en un objeto de tiempo real, y format-timestring, que permitió estructurar la cadena de texto de salida en el formato solicitado (Año-Mes-Día Hora:Minutos:Segundos).

BITACORA DE ERRORES

Requerimiento 1

Descubrimos que los nombres de los estados (los colores del semáforo) estaban escritos de distintas maneras en el código. En algunas partes usábamos un formato específico (como 'en-rojo o 'en-verde) y en otras partes usábamos palabras simples sin ese formato.

Como el programa no reconocía que se trataba de lo mismo, se confundía y fallaba al procesar y comparar. Debido a esto, nos aseguramos de usar la misma forma de escribir los estados en todo el código.

Requerimiento 2

```
(defun ubicar-fase (resto)
  (or (and (< resto 90) 'en-rojo)
      (and (>= resto 90) (< resto 210) 'en-verde)
      'en-amarillo))

(defun timer (tiempo-unix)
  (if (integerp tiempo-unix)
      (ubicar-fase (mod tiempo-unix 216))
      "Error: El tiempo Unix debe ser un numero entero"))
```



```

[1]> (Load "C:\\Codigos clisp\\timer")

;; Loading file C:\\Codigos clisp\\timer.lsp ...
;; Loaded file C:\\Codigos clisp\\timer.lsp
T
[2]> (timer 1781329205)

EN-VERDE
[3]> (timer 1781329205)

EN-VERDE
[4]> (timer 1781329300)

EN-VERDE
[5]> (timer 1781329410)

EN-VERDE
[6]> (timer "ahora")

>Error: El tiempo Unix debe ser un numero entero"
[7]> |

```

Se utilizó or para listar opciones, pero al ejecutarlo, el sistema tomaba 'en-verde como una condición siempre verdadera y por eso fallaba el resultado. Cambiamos a un cond, ya que esta estructura evalúa cada condición en orden y se detiene exactamente donde corresponde, permitiendo que la lógica funcione correctamente.

Requerimiento 3

```

[1]> (Load "C:\\Codigos clisp\\pruebaprueba")

;; Loading file C:\\Codigos clisp\\pruebaprueba.lsp ...
;; Loaded file C:\\Codigos clisp\\pruebaprueba.lsp
T
[2]> (informe 1718254800 'en-rojo 'en-verde)
Fecha 1718254800: la luz ha cambiado de EN-ROJO a EN-VERDE
NIL
[3]> |

```

```

[1]> (Load "C:\\Codigos clisp\\pruebaprueba")

;; Loading file C:\\Codigos clisp\\pruebaprueba.lsp ...
;; Loaded file C:\\Codigos clisp\\pruebaprueba.lsp
T
[2]> (informe 1718254800 'en-rojo 'en-verde)
Tiempo 1718254800: la luz ha cambiado de EN-ROJO a EN-VERDE
NIL
[3]> |

```

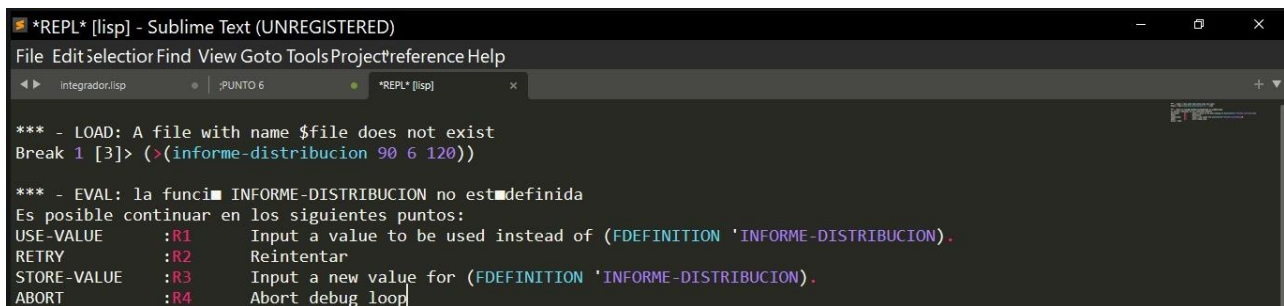
Error en los nombres que guarda la función con respecto a los requeridos por la consigna

Requerimiento 4

```
(defun duracion-ciclo ()
  (cond
    ((or (not (numberp segundos)) (<= segundos 0)) "Error: La duracion ingresada debe ser un numero positivo")
    ((and (>= segundos 35) (<= segundos 150)) (list segundos 'Rango-Optimo))
    ((< segundos 35) (list segundos 'Duracion-baja))
    (t (list segundos 'Duracion-alta))))
```

En el requerimiento 4, la función no operaba correctamente debido a la falta de parámetros. Corregimos colocando “segundos” como parámetros.

Requerimiento 6



```
*REPL* [lisp] - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Reference Help
integrador.lisp | PUNTO 6 | *REPL* [lisp] x
*** - LOAD: A file with name $file does not exist
Break 1 [3]> (> (informe-distribucion 90 6 120))
*** - EVAL: la función INFORME-DISTRIBUCION no está definida
Es posible continuar en los siguientes puntos:
USE-VALUE      :R1      Input a value to be used instead of (FDEFINITION 'INFORME-DISTRIBUCION).
RETRY          :R2      Reintentar
STORE-VALUE    :R3      Input a new value for (FDEFINITION 'INFORME-DISTRIBUCION).
ABORT          :R4      Abort debug loop
```

Al calcular la distribución temporal de los estados, operaciones aritméticas de Lisp como (/ (* 90 3600) 216) devolvían números racionales exactos (fracciones). Al intentar imprimir estos resultados mediante la directiva ~f (diseñada exclusivamente para números flotantes), Lisp arrojaba un error de tipado interrumpiendo la ejecución, por ende se aplicó una conversión explícita utilizando la función float sobre los cálculos matemáticos, de esta manera se aseguró que el flujo de datos entregara un valor decimal.

RESPUESTAS TEORICAS: FASE 3

Scheme es un dialecto minimalista del lenguaje Lisp, diseñado en los años 70 en el MIT por Guy L. Steele y Gerald Jay Sussman. A diferencia de Common Lisp, que es un lenguaje extenso y multiparadigma, Scheme se centra en la elegancia conceptual y la simplicidad sintáctica.

Industrias y Áreas de Uso: Se utiliza principalmente en el ámbito académico y de investigación (es el lenguaje base del famoso libro SICP), en sistemas embebidos, y como lenguaje de extensión o scripting para otras aplicaciones.

Empresas conocidas: Cisco (utiliza Scheme en procesos de control de calidad y herramientas internas), Google (históricamente en App Inventor), y la comunidad de software libre a través de GIMP (utiliza una variante llamada TinyScheme para sus plugins Script-Fu).

1. Lisp-1 vs Lisp-2: Espacios de Nombres

En Common Lisp (un "Lisp-2"), los símbolos tienen espacios de nombres distintos para variables y funciones. Por esto, cuando pasamos una función como argumento, necesitamos el operador `#` (abreviatura de function) para decirle al intérprete: "busca en el espacio de funciones", y luego usar `funcall` para ejecutarla.

En Scheme (un "Lisp-1"), existe un único espacio de nombres. Si un símbolo está ligado a un objeto función, se evalúa directamente como tal. No hace falta `#` porque no hay ambigüedad entre una variable y una función con el mismo nombre; y no hace falta `funcall` porque Scheme evalúa la primera posición de una lista y, si es una función, la ejecuta de inmediato.

2. Estructuración para Optimización de Llamada de Cola (TCO)

Para asegurar que el compilador de Scheme no agote la pila de memoria (Stack Overflow), se estructuró la recursividad de forma que la llamada recursiva sea la última acción realizada por la función.

En una función recursiva de cola (Tail Recursive), el resultado de la llamada recursiva se devuelve directamente sin realizar operaciones pendientes (como sumas o construcciones de listas posteriores). Esto permite que el sistema reutilice el marco de pila actual en lugar de crear uno nuevo, garantizando una eficiencia de memoria equivalente a un bucle imperativo [Fase 1].

La experiencia con Scheme nos permitió valorar un lenguaje que obliga a pensar en términos de inmutabilidad y recursividad desde su diseño. Aunque la sintaxis es similar a Common Lisp, la transición a un modelo "Lisp-1" simplifica la composición de funciones al tratar a estas como ciudadanos de primera clase sin necesidad de operadores especiales de acceso. El mayor desafío fue asegurar que toda iteración se transformara en recursividad de cola para aprovechar la TCO (*Tail Call Optimization*) obligatoria por especificación.

TCO Definición: Es una técnica que permite al compilador o intérprete procesar funciones recursivas de manera que no agoten la pila de memoria (evitando el error de Stack Overflow).

Funcionamiento: Para que la TCO ocurra, la llamada recursiva debe ser la última acción que realice la función. Al ser el último paso, el sistema no necesita "recordar" dónde se quedó ni guardar estados intermedios, por lo que puede reutilizar el marco de pila actual en lugar de crear uno nuevo para cada llamada.

Importancia en el Paradigma Funcional: Dado que en este paradigma se prohíben los bucles imperativos como loop o while, la recursividad de cola es la herramienta fundamental para realizar iteraciones o procesos largos de forma eficiente y segura.

Diferencia técnica: En una recursividad simple (como la factorial común), queda una operación pendiente (una multiplicación o suma) después de la llamada recursiva. En una recursividad de cola (TCO), el resultado de la llamada recursiva se devuelve directamente como el resultado final de la función, sin cálculos adicionales externos.

CONCLUSION

La realización de este proyecto integrador fue un desafío, desde la organización hasta la ejecución de las tareas asignadas. Realizar un trabajo con muchos requerimientos y detalles, nos obliga a adquirir nuevos conocimientos y adoptar diferentes pensamientos algorítmicos.

Las restricciones y las consideraciones permitió optar por otras alternativas que cumplen con lo pedido en los requerimientos, el proceso de clasificación taxonómica de cada función desarrollada sirvió para comprender en detalle el impacto en memoria de nuestras soluciones y la importancia de la optimización. Asimismo, la bitácora de depuración evidencia que los errores conceptuales cometidos, pero que fueron de gran ayuda para afianzar el entendimiento.

Por último, el análisis comparativo de la Fase 3 nos brindó una perspectiva diferente, se pudo observar la importancia de adaptar los conocimientos a otros lenguajes y a los desafíos a futuro.

Bibliografía:

- <https://www.scheme.org/>
- <https://academia-lab.com/enciclopedia/esquema-lenguaje-de-programacion/>